

SocialOrc — MVP Activation & End-to-End Verification Report

Date: 13 September 2026 **Repository:** SocialOrc (branch `main`) **Baseline commit:** `22d022d` — "fix(mvp): never report a publish that did not happen" **Report commit:** `4d0aee5` — "chore(verify): add activation verification scripts" **Production:** <https://www.socialork.com> **Verdict:** Production integrity **HOLDS**. Machine-runnable activation is **BLOCKED on credentials and external review**, with one credential pair proven invalid and two proven valid.

0. Executive summary

The MVP is **honest and internally safe**, and the previous pass has not regressed. What remains is almost entirely *external configuration*, not code.

Five findings matter, in order of consequence:

#	Finding	Consequence
1	Credentials for AI, X and TikTok exist in the local <code>.env</code> but were never added to Vercel Production. Production still runs on the original 14 variables.	No code change needed — push the vars. Immediate activation.
2	The X/Twitter credential pair is invalid. X's token endpoint returns <code>unauthorized_client</code> .	Pushing it to Vercel would not work. Fix in the X portal first.
3	TikTok credentials are valid (<code>invalid_grant</code> = client authenticated).	Safe to push; publishing still needs an account + audit.
4	AI is genuinely working — real DeepSeek completions on all three AI paths, <code>usedMock: false</code> .	Immediate activation once the key reaches Vercel.
5	E2E and durability suites cannot run at all — Docker's containerd content store returns <code>input/output error</code> .	Test-environment fault, independent of the repo. Needs a VM rebuild.

No production data was created, changed or deleted. No public post was published. No secret value was printed, logged or committed.

1. Scope and method

Objective: move the project from "*code ready, services unconfigured*" to "*operational and verified end-to-end wherever credentials are available*", without fabricating any success.

Method:

1. Inspect the repository, git state, Vercel environment and `.env` **before** changing anything.

2. Verify each phase only where a real credential exists; otherwise mark **BLOCKED**.
3. Prefer probes with **no side effect** — read-only API calls and deliberate-failure token requests — over actions that create public content.
4. Treat "*HTTP 200*" as insufficient evidence. Verify returned identifiers and provider-authentication semantics.
5. Run read-only database checks; never reset, truncate or delete.

Explicitly not done: no new features, no architectural change, no reimplementation of working code, no bypass of Meta App Review, Google verification or the TikTok audit.

2. Environment readiness

Inspected Vercel Production environment (names and scopes only) and the local `.env` (presence and value *length* only). **No secret value is reproduced anywhere in this report.**

2.1 Production (Vercel) — 14 variables, all Production scope

```
GOOGLE_CLIENT_ID      GOOGLE_CLIENT_SECRET  LINKEDIN_CLIENT_ID
LINKEDIN_CLIENT_SECRET  MOCK_SOCIAL_ADAPTERS  ALLOW_PUBLIC_SIGNUP
APP_URL                NEXTAUTH_URL          SIGNUP_ALLOWLIST
DIRECT_URL              CRON_SECRET            TOKEN_ENCRYPTION_KEY
NEXTAUTH_SECRET         DATABASE_URL
```

2.2 Readiness matrix

Service	Production	Local <code>.env</code>	Verification outcome
Blob	missing	empty (<code>""</code>)	NOT CONFIGURED → <code>MEDIA_STORAGE_NOT_CONFIGURED</code>
OpenAI	missing	key + model + base URL	CONFIGURED LOCALLY — WORKING ; production inert
LinkedIn	configured	configured	CONFIGURED — 4 real posts published
Telegram	missing	empty (<code>""</code>)	NOT CONFIGURED
X / Twitter	missing	id + secret present	CREDENTIALS INVALID
Facebook	missing	empty (<code>""</code>)	NOT CONFIGURED
Instagram	missing	empty (<code>""</code>)	NOT CONFIGURED
YouTube	missing	empty (<code>""</code>)	NOT CONFIGURED
TikTok	missing	key + secret present	credentials VALID ; production inert

2.3 The central gap

The AI, X and TikTok credentials are present **locally only**. They were never added to the Vercel Production environment, so the deployed application has never seen them. This is a configuration-deployment gap, not a code defect — nothing in the repository needs to change to close it.

3. Verification performed

Phase 1 — Vercel Blob / media storage → BLOCKED

`BLOB_READ_WRITE_TOKEN` is absent in both environments. Per the task rules, **no alternative storage fallback was implemented**. Media upload in production returns an honest refusal:

```
503 { "success": false, "error": "Media storage is not configured",
      "code": "MEDIA_STORAGE_NOT_CONFIGURED" }
```

No media row exists in the database (`MediaAsset` count = 0), and no `data: URL` is stored anywhere. There is nothing to migrate and nothing faking success.

Phase 2 — OpenAI / AI path → LIVE VERIFIED (local)

Executed against the real provider with the local key. The provider is reached through the OpenAI-compatible `OPENAI_BASE_URL` override:

```
provider host      : api.deepseek.com
model              : deepseek-chat
isOpenAIAvailable() : true

aiStudioGenerate  -> usedMock: false   variants: 2    1834 ms   unique texts 2/2
improveContent    -> isMock:   false           1368 ms
cascadeContent    -> adaptations: 2       2618 ms
```

Interpretation: `usedMock: false` on all three paths, real network latency, and distinct output per platform. The deterministic mock generator was **not** used. The AI subsystem is functional.

Production has no key and therefore returns `503 AI_NOT_CONFIGURED` rather than serving fabricated text — the designed honest failure. `mockAIAllowed = false` is reported live.

Model was `deepseek-chat` via a DeepSeek base URL, not OpenAI. Recorded because `OPENAI_MODEL` and `OPENAI_BASE_URL` are load-bearing here.

Phase 3 — LinkedIn → LIVE VERIFIED (historical), not re-published

Deliberately did **not** publish a new post: that would create permanent public content on a real profile without being asked. Verification was therefore limited to connection state, adapter/API contract tests, and existing publication evidence.

```
account      : Mohamed MATRAB
isActive    : true      needsReconnect: false
token expires : 2026-11-11T17:14:51.967Z
live token call: NOT POSSIBLE from this shell
reason       : stored tokens are encrypted with the PRODUCTION
               TOKEN_ENCRYPTION_KEY; this shell has a different one
```

The decryption failure is **correct security behaviour**, not a defect — a local development shell cannot read production OAuth tokens. It is reported rather than worked around.

Evidence the integration is real, from the production database:

```
4 PUBLISHED LinkedIn posts, every one carrying a real platformPostId (urn:li:share:...)
0 PUBLISHED posts with a null platformPostId
```

Published-identifier structure was verified; success was **not** inferred from an HTTP status code.

Phase 4 — Scheduling → CODE VERIFIED

Check	Result
Cron route exists (/api/cron/publish)	✓
Cron registered (*/5)	✓
Cron authentication enforced	✓ live request without auth returns 401
Pending jobs selected	✓ code path verified
Attempts increment at claim time	✓
Success sets platformPostId	✓
Failure follows retry/backoff logic	✓ 30 s → 1 min, capped 15 min
Stale PROCESSING recovery	✓ unit-tested (recoverStaleProcessingJobs , 10-min lease)
Dead-letter behaviour	✓ after MAX_ATTEMPTS = 3

The scheduler was **not modified** — no defect was found in it.

Live state: pendingJobs: 0 , PROCESSING orphans: 0. The historical FAILED LinkedIn job is the known dead-letter from the retired 20240601 API version, already corrected by the version bump to 202608 .

Not exercised: a scheduled post was not driven all the way to a live publish, because that requires a real target platform and would create public content.

Phase 5 — Telegram → BLOCKED

TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID are both empty/missing. No connection, no publish, and no simulated result. Reported as blocked exactly as instructed. The unsupported-video refusal (UNSUPPORTED_MEDIA) is covered by unit tests but was not exercised against a live bot.

Phase 6 — X / Twitter → BLOCKED — CREDENTIALS REJECTED

This phase produced the most important verification result of the task, because a naive check would have passed it.

Step 1	authorize URL -> x.com error string present? naive verdict	HTTP 200 no "credential ACCEPTED"
Step 2	POST https://api.x.com/2/oauth2/token with a deliberately bogus code error definitive verdict	HTTP 400 unauthorized_client CLIENT CREDENTIALS REJECTED

The authorize page returned a clean HTTP 200 with no error while the credential pair is not usable. A status-code-only verification would have reported a false pass and the user would have wired broken credentials into production. The token-endpoint probe is the decisive check: `invalid_client` / `unauthorized_client` means the client is not authenticated; `invalid_grant` means it is, and only the fake code was bad.

Interpretation: the client is not authorised for the `authorization_code` grant. This is not fixable in code — it requires enabling OAuth 2.0 user authentication (and the correct client type) for the app in the X developer portal.

Publishing state: no X account connected, so no tweet was attempted. The intentional text-only restriction and the `UNSUPPORTED_MEDIA` refusal for attached media are covered by unit tests.

Phase 7 — Facebook → BLOCKED

`FACEBOOK_APP_ID` / `FACEBOOK_APP_SECRET` are missing. No app credentials exist, so **no OAuth flow and no publish were attempted**. Even with credentials, `pages_manage_posts` requires **Advanced Access after Meta App Review**, which was confirmed still in force. No attempt was made to bypass that review.

Phase 8 — Instagram → BLOCKED

`INSTAGRAM_APP_ID` / `INSTAGRAM_APP_SECRET` are missing, and `instagram_content_publish` likewise requires **App Review**. No account connected; nothing published. The MVP's single-JPEG capability and honest rejection of multiple images are enforced in code and tests but were not exercised against the live API.

Phase 9 — YouTube → BLOCKED

`YOUTUBE_CLIENT_ID` / `YOUTUBE_CLIENT_SECRET` are missing. The existing resumable-upload implementation was **not rewritten**. Even with credentials, `youtube.upload` is a **restricted scope requiring Google verification**. No public test was possible.

Phase 10 — TikTok → credential LIVE VERIFIED, publishing BLOCKED

Using the same decisive method:

```
POST https://open.tiktokapis.com/v2/oauth/token/
  with a deliberately bogus code
HTTP 200
error    invalid_grant
verdict  CLIENT CREDENTIALS VALID – TikTok authenticated the client and
        rejected only the fake code
```

The `client_key` / `client_secret` pair is genuinely valid and safe to deploy.

Publishing remains blocked: no TikTok account is connected, and the application is unaudited, which restricts posts to `SELF_ONLY`. Per instructions, the audit was not bypassed. `TIKTOK_REDIRECT_URI` is unset locally; the platform's default callback is used.

Phase 13 — Health endpoint → VERIFIED

Live production output:

```
status      : ok
environment : production
commit      : 22d022d
capabilities : mockAdaptersAllowed = false
              mockAIAIAllowed       = false
              aiConfigured           = false
              mediaStorage            = unconfigured
connectors   : LINKEDIN  configured=true  missing=[]
              TWITTER   configured=false  missing=[TWITTER_CLIENT_ID, TWITTER_CLIENT_SECRET]
              INSTAGRAM configured=false  missing=[INSTAGRAM_APP_ID, INSTAGRAM_APP_SECRET]
              FACEBOOK  configured=false  missing=[FACEBOOK_APP_ID, FACEBOOK_APP_SECRET]
              TIKTOK    configured=false  missing=[TIKTOK_CLIENT_KEY, TIKTOK_CLIENT_SECRET]
              YOUTUBE   configured=false  missing=[YOUTUBE_CLIENT_ID, YOUTUBE_CLIENT_SECRET]
              TELEGRAM  configured=false  missing=[TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID]
platforms    : configured 1 / total 7
worker       : pendingJobs 0
```

Every required distinction is present: configured vs not configured, mock allowed vs disabled, AI configured, media storage configured. **No unconfigured platform is reported healthy.** Each names its exact missing variables.

4. Files changed

File	Status
<code>scripts/verify-ai.ts</code>	new
<code>scripts/verify-connectors.ts</code>	new

No production code was changed during this task. The application side remains exactly as deployed in `22d022d`.

Both scripts are read-only probes. They print provider hostnames, models, statuses and error strings — **never a credential value** — and write nothing to any platform.

5. Real verified integrations

Only genuinely executed external operations are listed.

Integration	Level	Evidence
AI (local runtime)	LIVE VERIFIED	Real DeepSeek completions, <code>usedMock: false</code> , 3/3 paths, unique output
TikTok credentials	LIVE VERIFIED	Token endpoint authenticated the client (<code>invalid_grant</code>)
X credentials	VERIFIED INVALID	Token endpoint returned <code>unauthorized_client</code>
LinkedIn	LIVE VERIFIED (historical)	4 real posts with <code>urn:li:share: ids</code> ; healthy connection, no reconnect flag
Health endpoint	VERIFIED	Live JSON above
Database integrity	VERIFIED CLEAN	Section 7

Not claimed as verified: any *new* publish to any platform, any OAuth round-trip, any media upload.

6. Blocked integrations

Integration	Blocking condition
Blob / media storage	Credential missing in both environments
OpenAI in production	Key exists locally, not in Vercel Production
Telegram	Credentials missing/empty
X / Twitter	Credentials rejected by X ; no connected account
Facebook	Credentials missing + Meta App Review (<code>pages_manage_posts</code>)
Instagram	Credentials missing + Meta App Review (<code>instagram_content_publish</code>) + IG Business account
YouTube	Credentials missing + Google restricted-scope verification (<code>youtube.upload</code>)
TikTok publishing	No connected account + TikTok audit (<code>SELF_ONLY</code> restriction)
LinkedIn re-publish	Intentionally not exercised — would create public content
E2E / durability tests	BLOCKED BY TEST ENVIRONMENT — see section 8

7. Database safety

Read-only checks. No `INSERT` , `UPDATE` , `DELETE` , `TRUNCATE` , `DROP` or migration was executed against production.

```
Mock published posts (platformPostId LIKE 'mock_%')      : 0    ✓
Mock URLs          (platformPostUrl LIKE '%mock.%')      : 0    ✓
Real LinkedIn post IDs (urn:li:share:...)                : 4    ✓ intact
PUBLISHED posts on unconfigured platforms                : 0    ✓
PUBLISHED posts with no platformPostId                   : 0    ✓
Stuck PROCESSING jobs                                    : 0    ✓
PENDING jobs                                              : 0    ✓
Media rows storing a data: URL                            : 0    ✓
Total media rows                                          : 0
```

Per-platform distribution — only LinkedIn has ever published:

Platform	DRAFT	APPROVED	PUBLISHED	FAILED
LINKEDIN	3	1	4 (4 with real ids)	1
TWITTER	2	–	0	–
INSTAGRAM	2	–	0	–
FACEBOOK	2	–	0	–
TIKTOK	1	–	0	–
YOUTUBE	1	–	0	–
TELEGRAM	1	–	0	–

```
Production data deleted : 0
Database reset          : NO
```

8. Tests

```
unit      452 tests | 449 passed | 3 failed | 0 skipped
typecheck PASS (tsc --noEmit, exit 0)
lint      1 error | 17 warnings  -> all pre-existing, in files untouched by this work
build     PASS (compiled successfully, 47/47 pages) at 22d022d
E2E       BLOCKED BY TEST ENVIRONMENT
durability BLOCKED BY TEST ENVIRONMENT
```

Three unit failures — all pre-existing, none introduced:

Test	Cause
tests/unit/engagement-api.test.ts	Prisma driver adapter is sqlite while the schema provider is postgres; identical at baseline
timezone.test.ts	Machine-timezone assumption (Oslo vs UTC)
trend-agent.test.ts	Dashboard card not mounted in the test DOM

The single lint error is `react/no-unescaped-entities` at `src/app/(dashboard)/settings/accounts/page.tsx:337` — pre-existing, unrelated to MVP activation, and deliberately left untouched.

Why E2E and durability are blocked: the throwaway Postgres container's data directory is corrupted:

```
FATAL: could not open file "global/pg_filenode.map": Input/output error
```

A clean, separate container was attempted rather than destroying the shared one — the user's Snora Supabase stack runs on the same Docker VM. That failed at the storage layer:

```
rpc error: ... /var/lib/containerd/io.containerd.content.v1.content/blobs/sha256/...  
: input/output error
```

Docker's containerd content store is returning I/O errors, so **no new container can be created**. This is a VM-level fault, independent of the repository. Rebuilding it (`colima delete && colima start`) would destroy the user's running Snora containers and was therefore not performed. Production behaviour and the Prisma schema were **not** altered to make anything pass.

9. Test matrix

Phase	Integration	Status
1	Blob / media upload	BLOCKED — MEDIA_STORAGE_NOT_CONFIGURED
2	OpenAI (local)	LIVE VERIFIED
2	OpenAI (production)	BLOCKED — not in Vercel; honest AI_NOT_CONFIGURED
3	LinkedIn publish	NOT EXERCISED — connection + adapter verified; avoided public content
4	Scheduling	CODE VERIFIED
5	Telegram	BLOCKED — credentials missing
6	X / Twitter	BLOCKED — credentials rejected (unauthorized_client)
7	Facebook	BLOCKED — credentials + App Review
8	Instagram	BLOCKED — credentials + App Review
9	YouTube	BLOCKED — credentials + Google verification
10	TikTok publish	BLOCKED — valid credentials, no account + audit
13	Health endpoint	VERIFIED
14	Database	VERIFIED CLEAN
16	Unit / typecheck / build	PASS (3 pre-existing failures explained)
16	E2E / durability	BLOCKED BY TEST ENVIRONMENT

10. Acceptance criteria

Criterion	Status
Production cannot fake a successful publish	✓
Production cannot silently use MockAdapter	✓
Production cannot silently use mock AI	✓
Production media uses real HTTPS storage	🛑 BLOCKED — no Blob token; fails honestly
Unsupported media rejected before any platform API request	✓
Scheduling works	✓
Stale PROCESSING jobs recover	✓
LinkedIn remains functional	✓
Every configured platform can be tested honestly	✓
Every unconfigured platform reports BLOCKED / NOT CONFIGURED	✓
No production secrets committed	✓
Typecheck passes	✓
Build passes	✓
Tests passing or explained if blocked	✓

11. Risks and observations

1. **Broken credentials were about to reach production.** The X pair is invalid, and only the token-endpoint probe revealed it. Any "does the authorize URL load?" check gives a false pass. *Lesson recorded in the committed script.*
2. **Local credentials are not deployed credentials.** The `.env` holds working AI and TikTok secrets that production has never seen. Local success is not evidence of production capability.
3. **Blocking media blocks most platforms.** Without Blob, every image/video post is refused by design — the honest behaviour, but it means only text posting can work today.
4. **LinkedIn tokens expire 2026-11-11 with no refresh token.** All three connections will die in roughly 60 days and require manual re-authentication. This is the most likely near-term outage.
5. `ALLOW_PUBLIC_SIGNUP=true` is live in production with an `@example.com` allowlist. Intentional for testing, but it should be closed before real users arrive.
6. **Preview deployments have no environment variables** — all 14 are Production-scope only, so previews cannot exercise auth or the database.
7. `MOCK_SOCIAL_ADAPTERS` is present in production but intentionally inert: mocks are gated on `NODE_ENV`, so a stale `true` cannot fake a publish. Worth leaving as a permanent safety net.

8. **Docker is unhealthy on this machine**, which removes the project's E2E and durability coverage until the VM is rebuilt.

12. Recommended next steps

Ordered by effort-to-impact:

1. **Add `OPENAI_API_KEY` , `OPENAI_MODEL` , `OPENAI_BASE_URL` to Vercel Production.** Verified working locally; no review required; activates AI Studio, generate and cascade.
2. **Add `BLOB_READ_WRITE_TOKEN` .** Unblocks every image and video capability across LinkedIn, Telegram, Facebook, Instagram and X.
3. **Add `TIKTOK_CLIENT_KEY` / `TIKTOK_CLIENT_SECRET` .** Credentials verified valid. Add `TIKTOK_REDIRECT_URI` to pin the callback explicitly.
4. **Do NOT deploy the X credentials yet.** Enable OAuth 2.0 user authentication in the X developer portal, then re-run `scripts/verify-connectors.ts` to confirm before wiring them in.
5. **Create the Telegram bot** via @BotFather (~2 minutes). The last instant win available.
6. **Start the Meta app and Google verification submissions now.** Both are multi-day external queues; begin them while other work continues. One Meta app covers Facebook posting, Instagram posting and Facebook sign-in.
7. **Schedule LinkedIn re-authentication before 11 November 2026.**
8. **Close public signup** once real onboarding begins.
9. **Rebuild the Docker VM** (`colima delete && colima start`) when the Snora Supabase containers can be interrupted, to restore E2E and durability coverage.

13. Reproducing this verification

```
# AI provider reachability and mock detection
node --env-file=.env --import tsx scripts/verify-ai.ts

# Connector credential validity (read-only; never prints a secret)
node --env-file=.env --import tsx scripts/verify-connectors.ts

# Live deployment posture
curl -s https://www.socialork.com/api/health

# Repository gates
npm run typecheck && npm run test:unit && npx eslint src tests && npm run build
```

No secret values appear in this report. No production data was modified. No platform publish was performed.